Chapter 4

PHP Fundamentals

Outline

- Overview of PHP
- Fundamentals of programming in PHP
- Introduction to OOP in PHP
- On the lab session
  - How to start working with PHP
    - Installation: XAMP/WAMP, Editor
    - Writing first PHP program
    -

PHP overview

- PHP is a recursive acronym which stands for ‘Hypertext Pre-processor
  - Open source server side scripting language designed specifically for the web
  - Server side scripting - Used to generate dynamic web pages
  - Interpreted language, scripts are parsed at run time
  - Embedded directly in to HTML
  - Supports a wide-range of databases (20+ODBC)
  - support for talking to other services using protocols

Why we learn PHP?

- Easy to learn, setup, host
- Multipurpose
  - Can be used to develop simple to complex website, CRM portal, E-commerce, Image/video/audio/ processing, wide range DB support,
- High demand on job market
- PHP CMS's support
  - Word press (CMS), Magneto(E-commerce), Drupal (CMS+E-commerce)
- PHP frameworks, Laravel, Symphony, CodeIgniter, yii

Overview ....

- Structurally similar to c/c++
- Supports procedural and object oriented paradigm
- Each PHP script must be enclosed in the reserved PHP tag

```php
<?php
// Display greeting message
echo "Hello, world!";
?>
```

```html
<!DOCTYPE HTML>
<html>
<head>
    <title>PHP Application</title>
</head>
<body>
<?php
// Display greeting message
echo 'Hello World!';
?>
</body>
</html>
```

variables

- Declared as $var_name = value;
  - names must be preceded by $ sign
  - Are case sensitive

```php
<?php
$txt = "Hello World!";
$number = 10;

// Display variables value
echo $txt;
echo "<br>";
echo $number;
?>
```

Constants

- Used to define values that cant change while the script is running, such as configuration settings
- defined using ***defin()*** function which accepts two arguments
  - the name of the constant and its value

```php
<?php
// Defining constant
define("SITE_URL", "https://www.tutorialrepublic.com/");

// Using constant
echo 'Thank you for visiting - ' . SITE_URL;
?>
```

Data types

- PHP supports total eight primitive data types:
  - Integer $a = 123
  - Floating point $a = 1.23
  - String $a = 'Hello world!';
  - Booleans $show_error = true;
  - Array $colors = array("Red", "Green", "Blue");
  - Object $message = new greeting;
  - resource $link = mysql_connect("localhost", "root",

Conditional ....

```php
<?php
$today = date("D");
switch($today){
    case "Mon":
        echo "Today is Monday. Clean your house.";
        break;
    case "Tue":
        echo "Today is Tuesday. Buy some food.";
        break;
    case "Wed":
        echo "Today is Wednesday. Visit a doctor.";
        break;
    case "Thu":
        echo "Today is Thursday. Repair your car.";
        break;
    case "Fri":
        echo "Today is Friday. Party tonight.";
        break;
    case "Sat":
        echo "Today is Saturday. Its movie time.";
        break;
    case "Sun":
        echo "Today is Sunday. Do some rest.";
        break;
    default:
        echo "No information available for that day.";
        break;
}
?>
```

Conditional statements

```php
<?php
$d = date("D");
if($d == "Fri"){
    echo "Have a nice weekend!";
} elseif($d == "Sun"){
    echo "Have a nice Sunday!";
} else{
    echo "Have a nice day!";
}
?>
```

```php
<?php
$name = isset($_GET['name']) ? $_GET['name'] : 'anonymous';
echo $name;
?>
```

```php
<?php
$name = $_GET['name'] ?? 'anonymous';
echo $name;
?>
```

PHP Array

- **Indexed array** — An array with a numeric key.
- **Associative array** — An array where each key has its own specific value.
- **Multidimensional array** — An array containing one or more arrays within itself.

```php
<?php
// Define nested array
$contacts = array(
    array(
        "name" => "Peter Parker",
        "email" => "peterparker@mail.com",
    ),
    array(
        "name" => "Clark Kent",
        "email" => "clarkkent@mail.com",
    ),
    array(
        "name" => "Harry Potter",
        "email" => "harrypotter@mail.com",
    )
);
// Access nested value
echo "Peter Parker's Email-id is: " . $contacts[0]["email"];
?>
```

Loops

```php
<?php
$i = 1;
while($i <= 3){
    $i++;
    echo "The number is " . $i . "<br>";
}
?>
```

```php
<?php
$i = 1;
do{
    $i++;
    echo "The number is " . $i . "<br>";
}
while($i <= 3);
?>
```

```php
<?php
for($i=1; $i<=3; $i++){
    echo "The number is " . $i . "<br>";
}
?>
```

```php
<?php
$colors = array("Red", "Green", "Blue");

// Loop through count array
foreach($colors as $value){
    echo $value . "<br>";
}
?>
```

```php
<?php
$superhero = array(
    "name" => "Peter Parker",
    "email" => "peterparker@mail.com",
    "age" => 18
);

// Loop through superhero array
foreach($superhero as $key => $value){
    echo $key . " : " . $value . "<br>";
}
?>
```

Functions

```php
<?php
// Defining function
function customFont($font, $size=1.5){
    echo "<p style=\"font-family: $font; font-size: {$size}em;\">Hello, world!</p>";
}

// Calling function
customFont("Arial", 2);
customFont("Times", 3);
customFont("Courier");
?>
```

Reading assignment:

- callback functions, anonymous functions, arrow functions

Class and objects

- Classes and objects are the two main aspects of object-oriented programming
- A class is a self-contained, independent collection of variables and functions which work together to perform one or more specific tasks,
  - while objects are individual instances of a class.
- A class is a template or blueprint from which individual objects can be created

Class

Register

Name

Abebe

Email

Username

Password

Register

Register

Name

Hadra

Email

Username

Password

Register

class

- A class can be declared using the class keyword, followed by the name of the class and a pair of curly braces ({})
- Once a class has been defined, objects can be created from the class with the ***new*** keyword.
- Class methods and properties can directly be accessed through pseudo-variable $***this*** object instance.
- The arrow symbol (->) is an OOP construct that is used to access contained properties and methods of a given object

Defining a class, properties and methods

```php
<?php
//Rectangle.php
class Rectangle
{
    // Declare  properties
    public $length = 0;
    public $width = 0;
```

```php
<?php
// Include class definition
require "Rectangle.php";

// Create a new object from Rectangle class
$obj = new Rectangle;

// Get the object properties values
echo $obj->length . "<br>";
echo $obj->width . "<br>";

// Set object properties values
$obj->length = 30;
$obj->width = 20;

// Read the object properties values again to show the change
echo $obj->length . "<br>";
echo $obj->width . "<br>";


// Call the object methods
echo $obj->getPerimeter() . "<br>";
echo $obj->getArea() . "<br>";
?>
```

Extending classes through inheritance

- Classes can inherit the properties and methods of another class using the extends keyword.
- Sub-classes inherit the methods and properties of the super class
  - Sub-classes can also override their definitions

```php
<?php

class User {

var $is_admin = false;

var $first_name;
var $last_name;
var $username;

function full_name() {
return $this->first_name . " " . $this->last_name;
}
}
```

```php
class Customer extends User {
var $city;
var $state; // or province
var $country;
function location() {
return $this->city . ", " . $this->state . ", " . $this->country;
}
}

class AdminUser extends User {
var $is_admin = true;

function full_name() {
return $this->first_name . " " . $this->last_name . " (Admin)";
}
}
```

Inheritance

```php
<?php
// Include class definition
require "Rectangle.php";

// Define a new class based on an existing class
class Square extends Rectangle
{
    // Method to test if the rectangle is also a square
    public function isSquare(){
        if ($this->length == $this->width){
            return true; // Square
        } else{
            return false; // Not a square
        }
    }
}

// Create a new object from Square class
$obj = new Square;

// Set object properties values
$obj->length = 20;
$obj->width = 20;

// Call the object methods
if($obj->isSquare()){
    echo "The area of the square is ";
} else{
    echo "The area of the rectangle is ";
};
echo $obj->getArea();
?>
```

Visibility modifiers

- Use to control access to properties and methods

Modifier	Accessible from
public	Anywhere
protected	Inside this class and its subclasses
private	Inside this class only

Magic methods

- Methods that are called in some circumstances , eg.
  - ➔ __construct() - executed automatically whenever a new object is created
  - ➔ __destruct() - executed automatically when the object is destroyed
- Must be marked public

Static ....

```php
<?php
class MyClass
{
    // Constructor
    public function __construct(){
        echo 'The class "' . __CLASS__ . '" was initiated!<br>';
    }

    // Destructor
    public function __destruct(){
        echo 'The class "' . __CLASS__ . '" was destroyed.<br>';
    }
}

// Create a new object
$obj = new MyClass;

// Output a message at the end of the file
echo "The end of the file is reached.<br>";
?>
```

```php
class Product {

  public $name;
  public $color;

  public function __construct() {
    Logger::log('Creating new product');
    $this->color = 'blue';
  }

}

$shirt = new Product;
echo $shirt->color;
// blue
```

Static properties and methods

- Behaviors related to the class generally
- Not tied to a particular instance
- Accessible directly from the class with out any instance
  - accessible without needing an instantiation of the class
- Defined using the keyword ***static***
- Static properties and methods can be accessed using the scope resolution operator (**::**), like
  - ClassName::$property and

Static....

```php
<?php
// Class definition
class HelloClass
{
    // Declare a static property
    public static $greeting = "Hello World!";

    // Declare a static method
    public static function sayHello(){
        echo self::$greeting;
    }
}
// Attempt to access static property and method directly
echo HelloClass::$greeting; // Output: Hello World!
HelloClass::sayHello(); // Output: Hello World!

// Attempt to access static property and method via object
$hello = new HelloClass;
echo $hello->greeting; // Strict Warning
$hello->sayHello(); // Output: Hello World!
?>
```

- Static properties and methods are inherited
- The keyword self in the example means "the current class".
- It is never preceded by a dollar sign ($) and
- always followed by the :: operator (e.g. self::$name).

Inherited static ….

- Static properties and methods are inherited
- Visibility modifiers function in the same way
- Inherited static properties are shared variables
- Changes to the parent values changes the subclass values
- Changes to a subclass value change the parent value
-

Static....

```
class Student {

  public static $grades = ['Freshman', 'Sophomore', 'Junior', 'Senior'];

}

class PartTimeStudent extends Student {
}

echo PartTimeStudent::$grades[0];
// Freshman

PartTimeStudent::$grades[] = 'Alumni';

echo implode(', ', Student::$grades);
// Freshman, Sophomore, Junior, Senior, Alumni
```

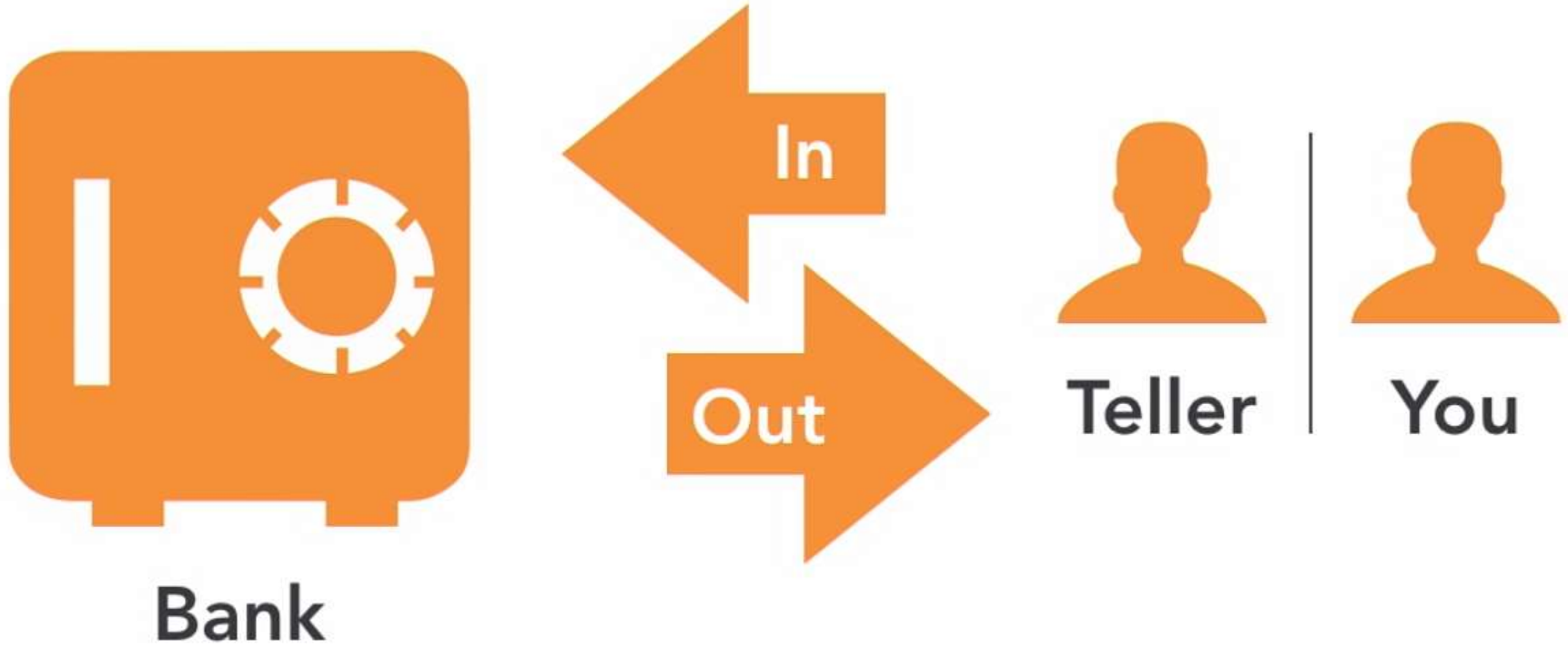
Class constants

- used for class values that do not change
- Values may contain expression
- Defined by using the keyword ***const***
- Support for visibility modifier since PHP 7.1
- Referenced with
  - ClassName::constantName of self:: constantName

Contants….

```
class Clock {

  public const DAY_IN_SECONDS = 60 * 60 * 24;

  public function tomorrow() {
    return time() + self::DAY_IN_SECONDS;
  }

}

echo Clock::DAY_IN_SECONDS;
// 86400

$clock = new Clock;
echo $clock->tomorrow();
// 1506456287
```

Setter and getter methods



tter & getter..

```
class Product {
  private $name;

  public function set_name($value) {
    $this->name = $value;
  }

  public function get_name() {
    return $this->name;
  }
}

$p = new Product;
$p->set_name('Birdhouse');
$p->get_name();
// Birdhouse
```

Setter & getter…

- Allow access to private properties
  - Through public methods
- Useful to regulate access
- Useful for read-only and write-only properties
- Useful for pre-processing values
- Avoid “naïve setter” and “naïve getter” methods
  - Like the example in the previous slide

```php
class Product {
  private $price;

  public function set_price($value) {
    $no_format = preg_replace('/[\$,]/', '', $value);
    $float = floatval($no_format);
    if($float <= 0) {
      trigger_error('Price must be greater than zero.', E_USER_NOTICE);
      return;
    }
    $this->price = $float;
  }

  public function get_price() {
    return '$' . number_format($this->price, 2);
  }
```

Next session Topics

- Forms
- Session and cookies
- Files basics
- Database
  - CRUD

Reading Assignment

- PHP strings
- Operators
- Late static binding
- Super global
- Autoloading